

PPTX RAG Service

Project Report — Architecture, Implementation, and Testing

A local-first Retrieval-Augmented Generation service for PowerPoint files, supporting both frontier (Claude API) and free local (Ollama) language models.

1. Project Overview

This project is a Retrieval-Augmented Generation (RAG) service that lets a user upload a PowerPoint (.pptx) file and ask natural-language questions about its contents. Unlike a simple search, the service reads and understands slide text, tables, and speaker notes, retrieves the most relevant pieces of content for a given question, and uses a language model to produce a grounded, cited answer — including the specific slide number the answer came from.

The service was built to satisfy a specific requirement: support both a frontier LLM (Claude, via the Anthropic API) and a local LLM (via Ollama, running entirely on the user's own machine, at no cost). Which one is used is controlled by a single configuration setting, so the same codebase works in both modes without any other changes.

1.1 What RAG Means Here

RAG stands for Retrieval-Augmented Generation. Instead of asking a language model to answer purely from what it was trained on (which risks hallucination — confidently making things up), the system first retrieves the actual relevant text from the uploaded document, then hands that text to the model as context and asks it to answer using only that context. This grounds every answer in real content from the specific file the user uploaded, and lets the model cite exactly which slide the information came from.

The pipeline has four stages, each handled by a separate file in the codebase:

- Extraction — pull structured text, tables, and notes out of the .pptx file
- Chunking & Embedding — break content into pieces and convert each into a numeric vector representation
- Retrieval — given a question, find the most relevant stored chunks by similarity
- Generation — pass those chunks to an LLM (Claude or a local Ollama model) to produce the final answer

2. Technology Stack

Component	Technology
Backend framework	FastAPI (Python) — serves HTTP endpoints for upload and question-answering
PPTX parsing	python-pptx — reads shapes, text frames, tables, and speaker notes from slides
Embeddings	sentence-transformers (all-MiniLM-L6-v2) — runs locally, free, no API cost or compiler dependency
Vector storage & search	Custom lightweight store using NumPy + pickle — no external vector database required
Frontier LLM	Claude (Anthropic API) — claude-sonnet-4-6, used when LLM_PROVIDER=anthropic
Local LLM	Ollama (llama3.1, 8B parameters) — runs entirely offline on the user's machine, free
Server	Uvicorn — ASGI server that runs the FastAPI app

Note on the vector store: the original design used ChromaDB, a popular vector database. During setup, ChromaDB's dependency (chroma-hnswlib) required a C++ compiler to install on Windows, which the target machine did not have. Rather than requiring the user to install Visual Studio Build Tools, the vector store was rewritten as a lightweight NumPy-based cosine-similarity search with results persisted to a local pickle file. This removed the compiler dependency entirely while keeping the same retrieval quality for the scale of a single-user, single-machine deployment.

3. Project Structure

The project is organized as follows:

```
rag_service/
├── app/
│   ├── __init__.py
│   ├── main.py
│   ├── extractor.py
│   ├── vectorstore.py
│   ├── vectorstore_utils.py
│   ├── llm.py
│   └── config.py
├── requirements.txt
├── .gitignore
└── README.md
```

The table below summarizes what each file is responsible for, before the detailed breakdown in Section 4.

File	Purpose
app/__init__.py	Empty file that marks the app/ folder as a Python package, so files can import from each other (e.g. from app.config import ...).
app/main.py	The FastAPI application. Defines the HTTP endpoints: /upload, /ask, /sources, /health.
app/extractor.py	Reads a .pptx file and extracts titles, body text, tables, and speaker notes into structured documents ready for indexing.
app/vectorstore.py	Handles chunking, embedding, storing, and searching content. This is the retrieval half of RAG.
app/vectorstore_utils.py	Small helper function for splitting long text into overlapping chunks.
app/llm.py	Sends retrieved context to either Claude or Ollama and returns the generated answer. This is the generation half of RAG.
app/config.py	Central settings file — model names, chunk size, which LLM provider to use, etc.
requirements.txt	List of all Python packages the project depends on, for pip install -r requirements.txt.
.gitignore	Tells Git which files/folders to exclude from version control (virtual environment, local data, secrets).
README.md	Setup and usage instructions for anyone else picking up the project.

4. Detailed File-by-File Breakdown

4.1 app/config.py (18 lines)

Purpose: a single place to hold every setting the rest of the app reads from, so nothing is hardcoded inside logic files.

What it defines:

- `LLM_PROVIDER` — either "anthropic" or "ollama"; read from an environment variable so it can be switched without editing code.
- `ANTHROPIC_MODEL` — which Claude model to call (claude-sonnet-4-6) when in frontier mode.
- `OLLAMA_BASE_URL` and `OLLAMA_MODEL` — where the local Ollama server runs and which local model to use (llama3.1).
- `EMBED_MODEL_NAME` — the sentence-transformers model used to turn text into vectors (all-MiniLM-L6-v2).
- `CHROMA_DIR` — folder path where the local vector store data is saved to disk.
- `CHUNK_SIZE` / `CHUNK_OVERLAP` — how long text chunks are (500 characters) and how much they overlap (80 characters), so a sentence split across two chunks isn't lost entirely.
- `TOP_K` — how many relevant chunks are retrieved per question (default 5).

4.2 app/extractor.py (139 lines)

Purpose: converts a raw .pptx file into clean, structured text ready to be embedded and searched. This is the most content-heavy file in the project because PowerPoint files store data in a way that needs careful handling.

Key functions:

- `_table_to_text(table)` — walks every row and cell of a PowerPoint table and renders it as readable pipe-delimited text (e.g. "Product | Price | Roast Level"), so table structure survives being converted to plain text.
- `_has_real_content(text)` — a filter that checks whether a piece of text has actual words or numbers, versus being blank. This was added after testing revealed that decks often contain empty, unfilled template tables (e.g. a blank schedule grid); indexing these blank tables was polluting search results with meaningless matches, so they are now skipped entirely.
- `extract_pptx(file_path)` — the main extraction function. Loops through every slide, then every shape on that slide: pulls the title, all text boxes, all tables (using the two helpers above), and any grouped shapes (shapes nested inside a group, which python-pptx does not expose directly by default). Also pulls speaker notes if present. Returns one structured dictionary per slide.
- `_find_boilerplate_lines(slides_data)` — scans across all slides and finds short lines of text that repeat on a large fraction of them (for example, a navigation bar reading "ABOUT OUR COMPANY | SERVICES | CLIENTS | PROJECTION" that appeared on nearly every slide of a template deck). These repeated lines add noise without adding information, and were found during testing to actively confuse the language model's reasoning about slide content, so they are automatically detected and removed.
- `slides_to_documents(slides_data, source_name)` — flattens each slide's extracted content into separate "documents": one for body text, one per table, one for notes (if present). Each document carries metadata (source filename, slide number, content type) so that later, when an answer is generated, the system knows exactly which slide to cite.

Why this design: keeping tables as separate documents from body text (rather than mixing them into one blob) was a deliberate choice — tables have a rigid row/column structure that gets diluted and harder for the model to reason about if it's mixed in with surrounding paragraph text.

4.3 app/vectorstore.py (109 lines)

Purpose: this is the retrieval engine — it takes the structured documents from the extractor, turns them into a searchable index, and finds the most relevant pieces of content for any given question.

Key parts of the VectorStore class:

- `__init__` — loads the sentence-transformers embedding model into memory, and loads any previously saved index from disk (if the service has been used before).
- `add_documents(docs)` — the indexing function. For each document, splits long body text into overlapping chunks (using the helper in `vectorstore_utils.py`) but keeps tables whole (since splitting a table mid-row would destroy its meaning). Converts every chunk into a numeric embedding vector using the sentence-transformers model, then stores it. Important behavior: if a file with the same name is uploaded again, its old chunks are deleted first before the new ones are added — this prevents duplicate content from stacking up in the index, which was found during testing to actively skew search results toward whichever chunks happened to be duplicated.
- `query(question, top_k, source)` — the search function. Converts the question into an embedding using the same model, then computes cosine similarity between the question's vector and every stored chunk's vector, returning the top-scoring matches. Two refinements were added after testing: (1) a slide-number detector — if a question explicitly names a slide ("explain slide 3"), that slide's chunks are boosted to guarantee they appear in the results, since semantic similarity alone does not reliably connect the words "slide 3" to a number like "03" appearing inside slide content; (2) a similarity floor — chunks scoring below a minimum relevance threshold are dropped rather than padding out the result list with noise.
- `list_sources()` — returns the list of all uploaded filenames currently indexed.

Why NumPy instead of a vector database: for the scale of this project (a single user, one machine, a handful of decks), a full vector database adds setup complexity and dependency weight without a meaningful performance benefit. Cosine similarity over a NumPy array is simple, has zero extra dependencies, and was fast enough in testing.

4.4 app/vectorstore_utils.py (13 lines)

Purpose: a small, standalone helper function, `chunk_text(text, size, overlap)`, that splits a long string into fixed-size overlapping pieces. Kept in its own file separate from `vectorstore.py` to avoid a circular import (`vectorstore.py` needs it, but putting it directly inside `vectorstore.py` created an import ordering conflict during testing).

4.5 app/llm.py (78 lines)

Purpose: the generation half of RAG — takes the chunks retrieved by `vectorstore.py` and asks a language model to write the final answer, grounded in that content.

Key parts:

- `SYSTEM_PROMPT` — the instructions given to the LLM before every question. It tells the model to answer only from the provided context, to present table-derived answers as a table, to always cite the slide number(s) the answer came from, and to avoid inventing numbers or names not present in the retrieved content.
- `build_context(hits)` — formats the retrieved chunks into a labeled block of text (showing source filename, slide number, and content type for each) so the model can see exactly where each piece of information came from.
- `_answer_with_anthropic(question, hits)` — sends the question and context to the Claude API using the official anthropic Python SDK, and returns the response text. Used when `LLM_PROVIDER` is set to "anthropic".
- `_answer_with_ollama(question, hits)` — sends the same question and context to a locally running Ollama server via a plain HTTP POST request (Ollama exposes a REST API on localhost:11434). Used when `LLM_PROVIDER` is set to "ollama" (the default). This is what allows the service to run completely free, with no API costs, once Ollama and a model are installed locally.

- `answer_question(question, hits)` — the single entry point the rest of the app calls; it checks the configured provider and routes to whichever of the two functions above is appropriate. This is the one place in the codebase that decides frontier vs. local — everything else in the pipeline is identical either way.

Testing note: the local model choice matters for accuracy. A smaller model (llama3.2, 1B parameters) was tested first for speed, but was found during testing to occasionally hallucinate numbers when reading dense data (for example, misreporting a 65%/27% gender split as 27%/36%). Switching to the larger llama3.1 (8B parameters) resolved this — it correctly read and reported the same data. This trade-off (smaller = faster but less reliable on precise numbers, larger = slower but accurate) is noted in the README for future reference.

4.6 app/main.py (68 lines)

Purpose: the FastAPI application itself — the file that ties every other module together into a running web service with HTTP endpoints.

Endpoints defined:

- `POST /upload` — accepts a .pptx file upload, saves it temporarily, calls `extract_pptx()` and `slides_to_documents()` from `extractor.py`, then calls `store.add_documents()` to index it. Returns the filename, number of slides extracted, and number of chunks indexed.
- `POST /ask` — accepts a JSON body with a question (and optional source filename filter and `top_k` count), calls `store.query()` to retrieve relevant chunks, then calls `answer_question()` from `llm.py` to generate the final answer. Returns the answer text plus a list of which slides/sources it drew from.
- `GET /sources` — returns the list of all currently indexed filenames, useful for confirming what has been uploaded.
- `GET /health` — a simple endpoint that returns `{"status": "ok"}`, used to confirm the server is running.

The interactive API documentation (Swagger UI) is auto-generated by FastAPI and available at `/docs`, which was used throughout testing to manually try uploads and questions without needing to write separate client code.

4.7 requirements.txt

Lists every Python package the project depends on, pinned to specific versions where it mattered for compatibility: `fastapi`, `uvicorn`, `python-pptx`, `sentence-transformers`, `numpy`, `anthropic` (the Claude SDK), `requests` (used for calling the local Ollama API), `python-multipart` (required by FastAPI for handling file uploads), and `pydantic` (used for validating the structure of incoming JSON requests).

5. End-to-End Data Flow

Walking through exactly what happens for one upload and one question:

5.1 Uploading a File

- User sends a .pptx file to POST /upload.
- main.py saves it to a temporary file and calls extract_pptx() (extractor.py).
- extract_pptx() opens the file with python-pptx, loops through every slide and shape, pulls out title/body/tables/notes, and filters out empty tables.
- slides_to_documents() converts the per-slide data into a flat list of documents, stripping any repeated boilerplate text detected across the deck.
- store.add_documents() (vectorstore.py) removes any old chunks from a file with the same name, splits long text into overlapping chunks, embeds every chunk with sentence-transformers, and saves the result to disk.
- The endpoint returns a summary: filename, slide count, chunk count.

5.2 Asking a Question

- User sends a question (plus optional filters) to POST /ask.
- store.query() (vectorstore.py) embeds the question, checks if a slide number was explicitly named, computes similarity against every stored chunk, and returns the top-scoring matches.
- build_context() (llm.py) formats those matches into a labeled context block.
- answer_question() sends the question and context to either Claude or the local Ollama model, depending on config.LLM_PROVIDER.
- The model's answer, along with the list of source slides it drew from, is returned to the user.

6. Testing and Issues Found During Development

The service was tested against two decks: a real-world 53-slide construction company profile template, and a custom-built 8-slide brochure deck created specifically for clean end-to-end testing (plain paragraphs, real tables, no ambiguous graphics). Several real issues were found and fixed during this process:

Issue Found	Fix Applied
chroma-hnswlib required a C++ compiler to install, which the machine did not have	Replaced ChromaDB with a lightweight NumPy + pickle vector store — no compiler dependency
Blank/empty template tables were being indexed and outranking real slide content in search results	Added a content check that skips tables with no real words or numbers before indexing
Repeated navigation text on every slide confused the model's understanding of slide content	Added automatic detection and removal of text lines repeated across a large fraction of slides
Re-uploading the same file created duplicate chunks, which crowded out other relevant content	Uploading a file with an existing name now replaces its old chunks instead of appending
Questions naming a specific slide ("explain slide 3") sometimes failed to retrieve that slide	Added explicit slide-number detection that guarantees the named slide's content is included
The 1B-parameter local model occasionally misread numbers from dense data (e.g. wrong percentages)	Switched default local model to the larger, more accurate 8B-parameter llama3.1
Ollama's response could exceed the original 120-second request timeout on slower hardware	Increased the timeout to 300 seconds for local model calls

One attempted fix was tested and reverted: an approach to explicitly pair standalone numbers with their labels (for infographic-style stat blocks) was tried, but verification showed it produced incorrect pairings for two-column layouts, so it was rolled back rather than shipped. This is documented as a known limitation rather than silently left broken.

7. Known Limitations

- Infographic-style stat callouts — slides where numbers are arranged visually in a grid or dashboard style (rather than in sentences or real tables) can have their number-to-label association lost during text extraction, since PowerPoint's XML stores shapes in document order, not visual position. Regular text, bullet points, and real tables are unaffected.
- Charts and images are not read — only text and table content is extracted; native PowerPoint charts and embedded pictures are not OCR'd or interpreted.
- Local model accuracy depends on model size — smaller local models are faster but less reliable on precise numeric data; llama3.1 (8B) was found to be a reasonable balance during testing.

8. Conclusion

The final service meets the original requirement: build a RAG service that can extract information from an uploaded PowerPoint, including data inside tables, and answer questions about it — using either a frontier LLM or a local LLM, selectable via configuration. The codebase is organized into clear, single-purpose modules (extraction, storage/retrieval, generation, and the API layer), tested against real and custom-built decks, and pushed to a GitHub repository for version control and future reference.